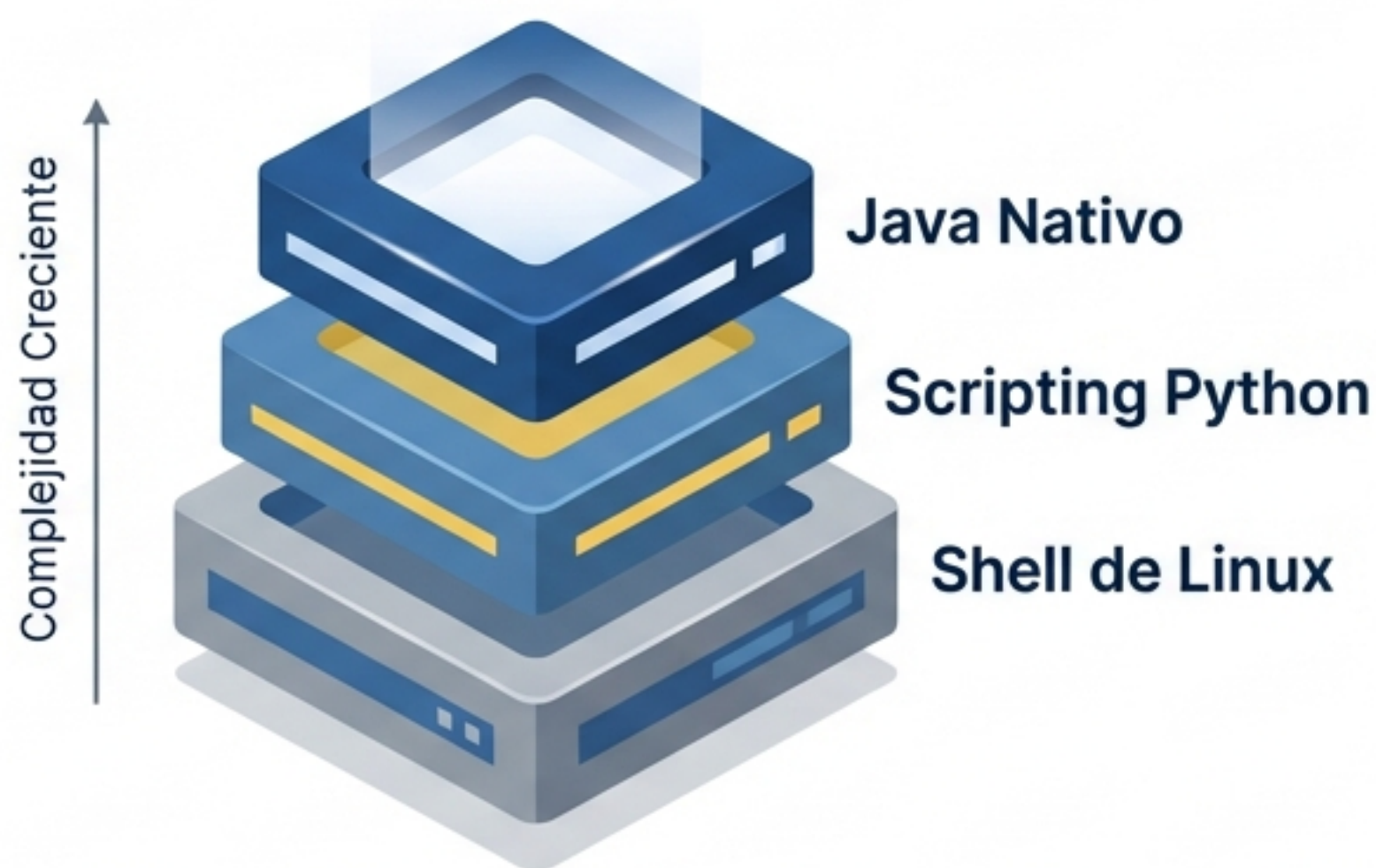


Pruebas de Concepto en Cluster Big Data

Guía práctica de ejecución: Shell, Python y Java MapReduce



1. Shell de Linux

Validación inmediata de infraestructura y E/S.

2. Scripting Python

Introducción de lógica externa y distribución de ficheros.

3. Java Nativo

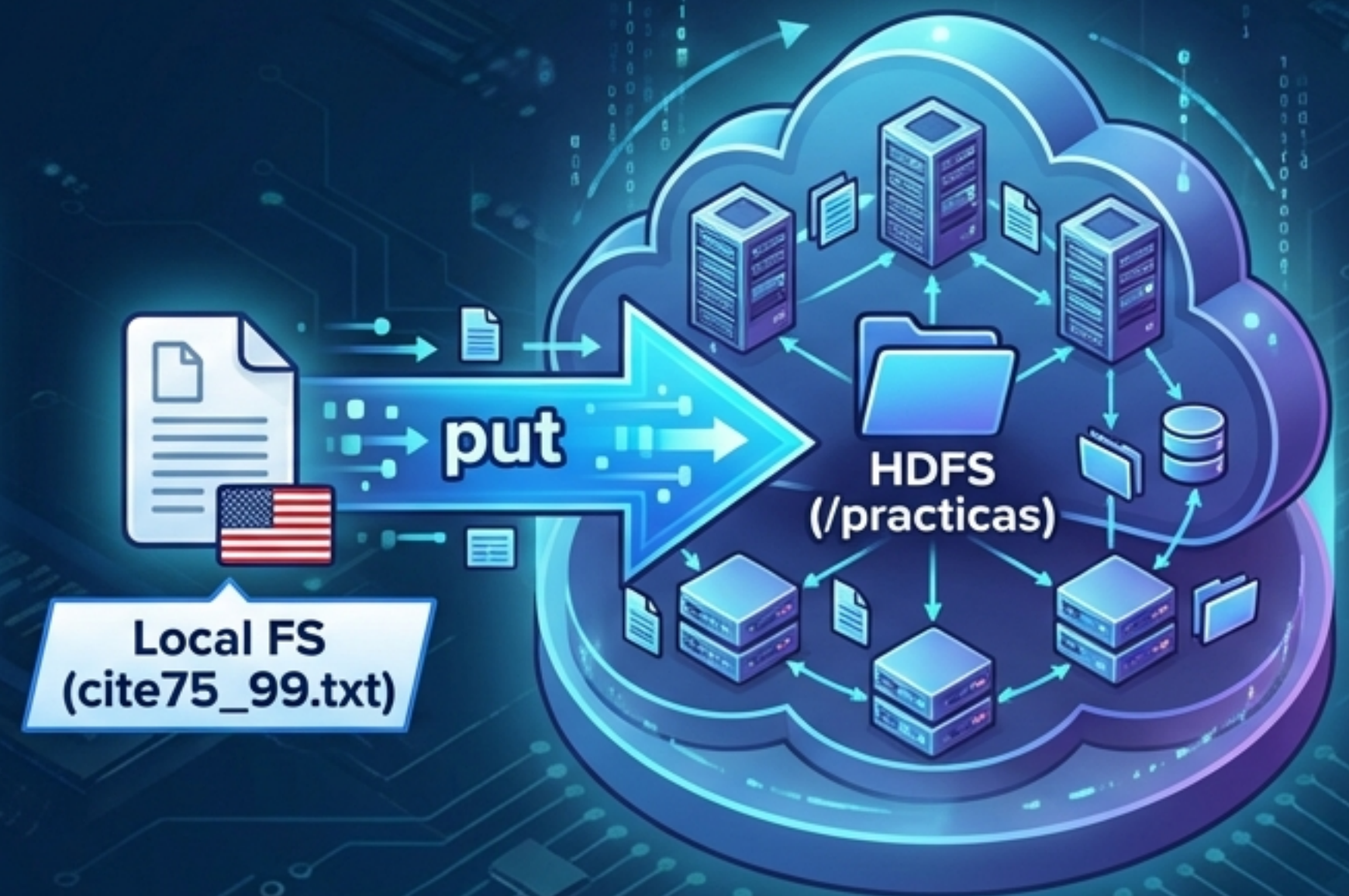
Compilación, empaquetado y despliegue empresarial.

Preparación del Dataset: Carga en HDFS

Antes de procesar, debemos poblar el ecosistema. Utilizaremos el fichero de patentes de Estados Unidos **cite75_99.txt**.

```
Terminal - HDFS Command  
  
hdfs dfs -put cite75_99.txt /practicass
```

Nota: Este comando valida la escritura en el sistema de archivos distribuido. Si falla, no continúe con MapReduce.



Nivel 1: Hadoop Streaming con Shell de Linux

El enfoque de menor fricción

Utilizamos la librería *Hadoop Streaming* para simular un programa MapReduce usando *pipes* de Unix. Esto nos permite validar la infraestructura sin compilar código complejo.



Objetivo: Extraer el segundo campo (patente citada) y eliminar duplicados para obtener una lista única.

Ejecución Nivel 1: Sintaxis del Comando

```
Terminal - HDFS Command

hadoop jar /opt/hadoop/share/hadoop/tools/
/lib/hadoop-streaming-2.9.0.jar \
-input /practicas/cite75_99.txt \
-output resultado8 \
-mapper 'cut -f 2 -d ,' \
-reducer 'uniq'
```

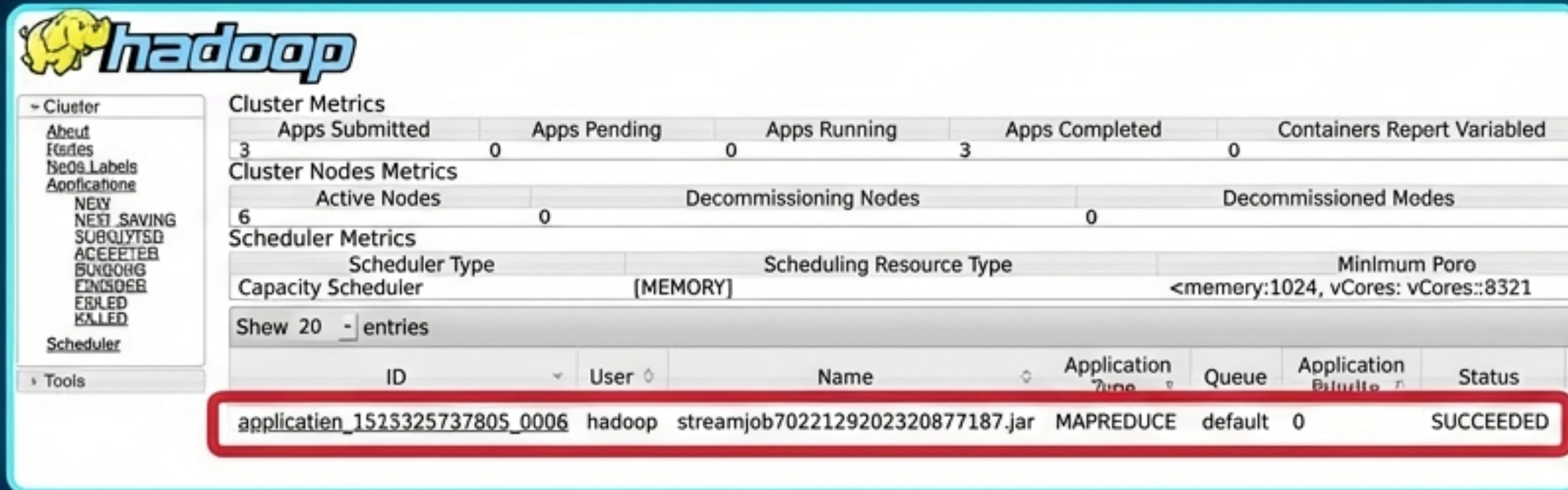
Ruta absoluta a la librería streaming (ajustar versión si es necesario).

El directorio destino **NO** debe existir previamente.

Comandos nativos de Linux entre comillas simples.

Monitoreo del Cluster en Tiempo Real (YARN)

Acceda a la interfaz web del Resource Manager (<http://nodo1:8088>) para verificar la asignación de contenedores.



The screenshot shows the Hadoop Resource Manager web interface. On the left is a navigation menu with options like 'Cluster', 'About', 'Nodes', 'Nodes Labels', 'Applications', and 'Scheduler'. The main area displays 'Cluster Metrics' with a table showing 3 Apps Submitted, 0 Pending, 3 Running, and 0 Completed. Below this are 'Cluster Nodes Metrics' and 'Scheduler Metrics'. At the bottom, a table lists applications, with one application highlighted in a red box: 'application_1525325737805_0006' by user 'hadoop', running 'streamjob7022129202320877187.jar' on the 'MAPREDUCE' queue, with a status of 'SUCCEEDED'.

Cluster Metrics					
Apps Submitted	Apps Pending	Apps Running	Apps Completed	Containers	Report Variated
3	0	3	0		

Cluster Nodes Metrics			
Active Nodes	Decommissioning Nodes	Decommissioned Nodes	
6	0	0	

Scheduler Metrics		
Scheduler Type	Scheduling Resource Type	Minimum Poro
Capacity Scheduler	[MEMORY]	<memory:1024, vCores: vCores::8321

ID	User	Name	Application	Queue	Application	Status
application_1525325737805_0006	hadoop	streamjob7022129202320877187.jar	MAPREDUCE	default	0	SUCCEEDED

Checklist de Éxito:

1. State: SUCCEEDED
2. Maps Completed: 2
(coincide con los bloques del fichero)
3. Reduces Completed: 1



The screenshot shows the 'ApplicationMaster' detail page for the application 'application_1525325737805_0006'. It shows attempt number 1, started on 'Sun Jan 07 15:08:32 CET 2018' on node 'nodo7:8042'. Below this is a table showing task counts: 2 Maps and 1 Reduce completed successfully. The 'Total' column shows 2 Maps and 1 Reduce. The 'Complete' column shows 2 Maps and 1 Reduce. The 'Attempt Type' table shows 0 Failed, 0 Killed, and 2 Successful. The 'Maps' and 'Reduces' rows show 0, 0, and 2, 1 respectively.

ApplicationMaster			
Attempt Number	Start Time	Node	Log
1	Sun Jan 07 15:08:32 CET 2018	nodo7:8042	logs

Task Type	Total	Complete
Map	2	2
Reduce	1	1

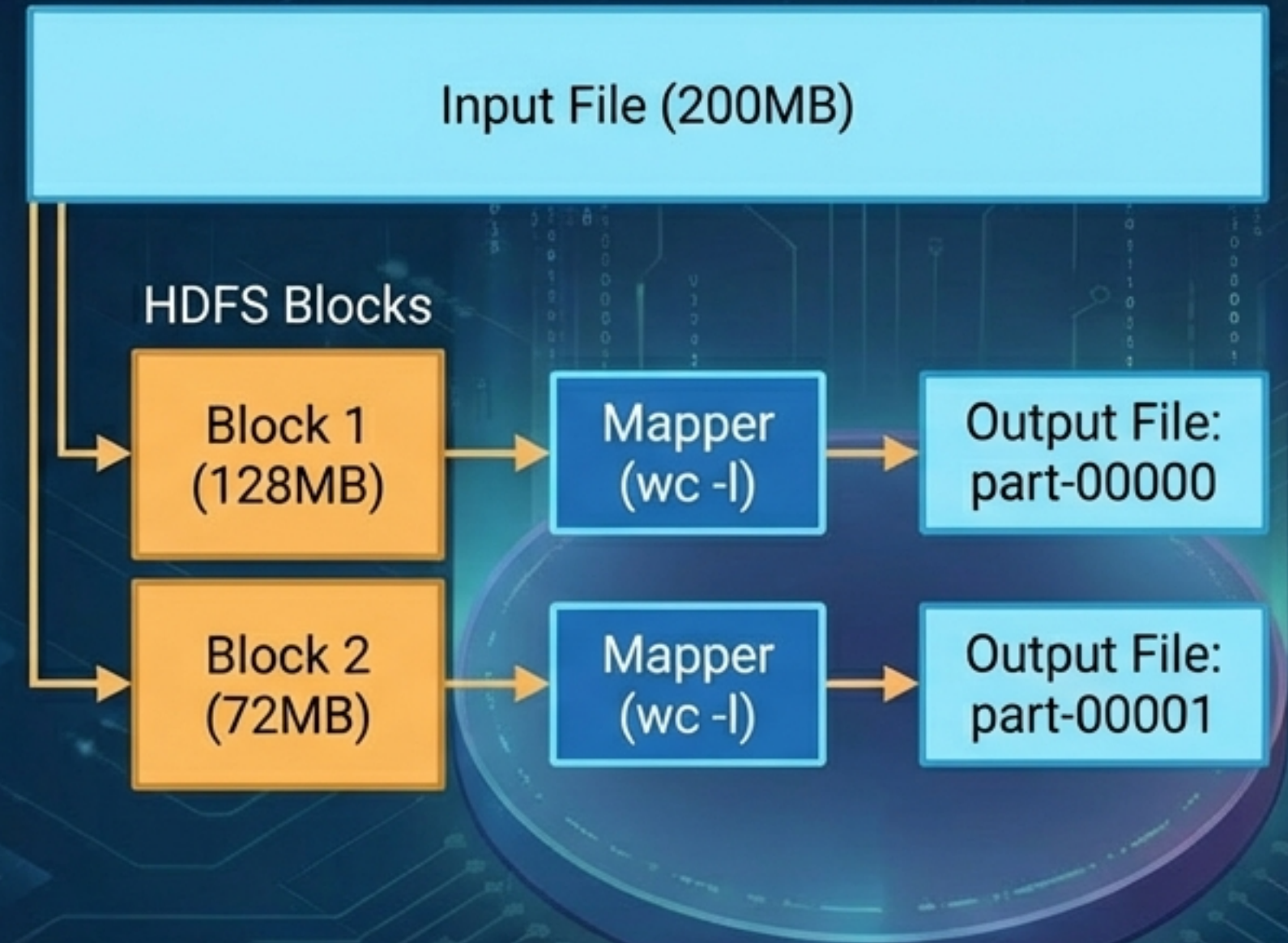
Attempt Type	Failed	Killed	Successful
Maps	0	0	2
Reduces	0	0	1

Variación 1B: Job 'Map-Only' (Sin Reducer)

Optimización y estructura de bloques

```
Terminal - HDFS Command

hadoop jar ...hadoop-streaming-2.9.0.jar \
-D mapred.reduce.tasks=0 \
-input /practicas/cite75_99.txt \
-output /resultado9 \
-mapper 'wc -l'
```



Al desactivar el Reducer (**-D mapred.reduce.tasks=0**), el resultado refleja la fragmentación física de HDFS. Obtenemos un fichero de salida por cada bloque de datos.

Nivel 2: Streaming con Python

Introduciendo lógica externa: `rand.py`

Sustituimos los comandos de Shell por un script Python que filtra filas aleatoriamente. Este paso verifica que el cluster puede ejecutar intérpretes de lenguajes de scripting.

`rand.py`

```
#!/usr/bin/env python
import sys, random
for line in sys.stdin:
    # Filtra el 10% de las líneas (aprox)
    if(random.randint(1,100) <= int(sys.argv[1])):
        print line.strip()
```



Ejecución Nivel 2: Distribución de Código



rand.py

-file rand.py
(Distribución automática al Distributed Cache)



```
hadoop jar ...hadoop-streaming-2.9.0.jar \  
-D mapred.job.reducers=1 \  
-input /practicas/cite75_99.txt \  
-output /resultado11 \  
-mapper 'rand.py 10' \  
-file rand.py
```

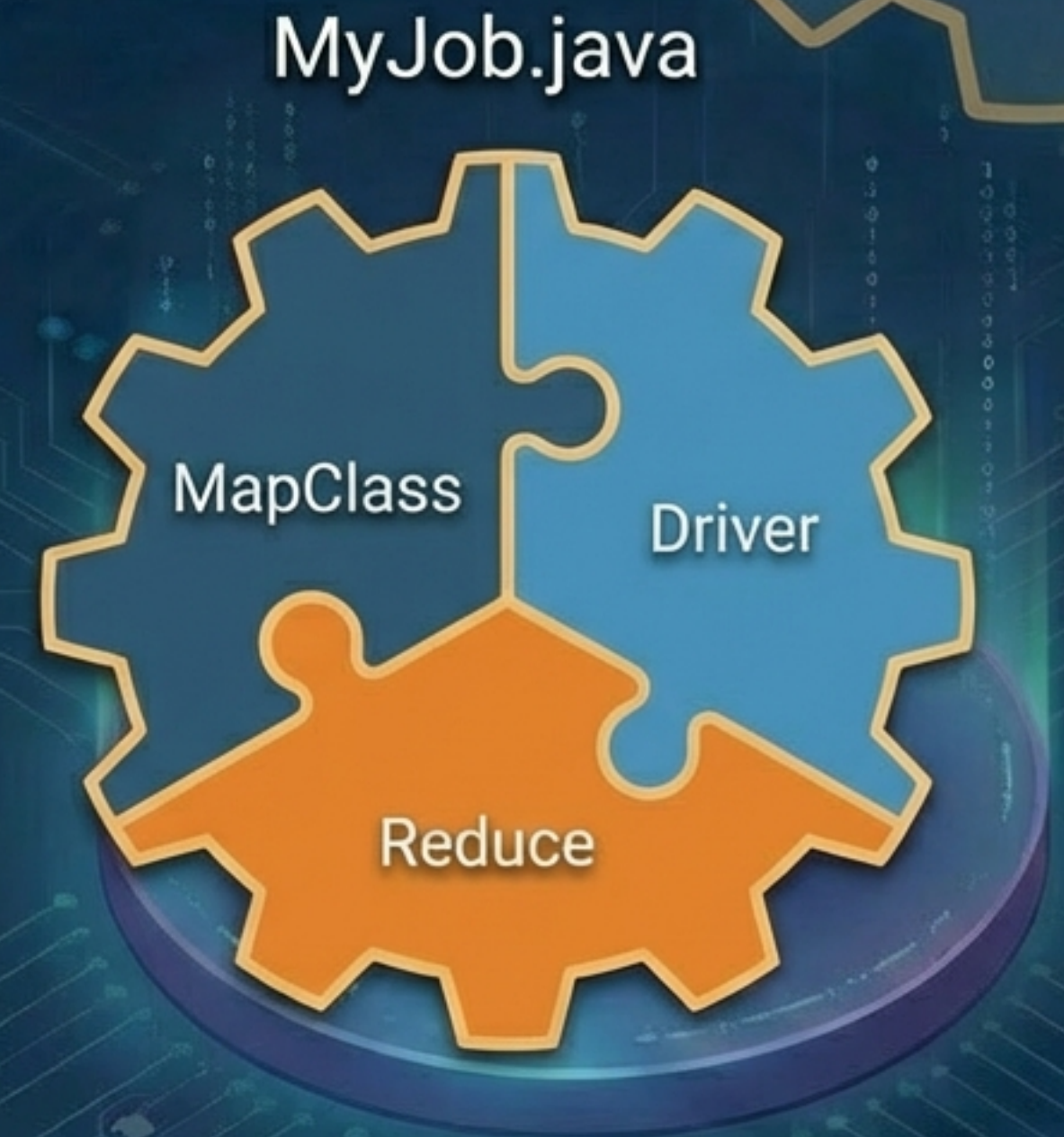
CRÍTICO: Este flag envía el script físico a los nodos de trabajo. Sin él, el Mapper fallará por "file not found".



Nivel 3: MapReduce Nativo en Java

El estándar empresarial

Objetivo	Método	Archivo Fuente
Agrupar patentes por código de cita principal.	Escribir una clase Java que implemente `Mapper`, `Reducer` y `ToolRunner`.	`MyJob.java`



Anatomía del Código: La Clase Mapper

```
public static class MapClass extends Mapper<LongWritable, Text, Text, Text> {  
  
    public void map(LongWritable key, Text value, Context context) ... {  
        // Divide la línea por comas  
        String[] citation = value.toString().split(",");  
  
        // Inversión: Clave = citation[1], Valor = citation[0]  
        context.write(new Text(citation[1]), new Text(citation[0]));  
    }  
}
```

La lógica clave está en `context.write`. Invertimos el orden para agrupar por el segundo campo del CSV original.

Anatomía del Código: Reducer y Configuración

```
public void reduce(Text key, Iterable<Text>
    values, Context context) ... {
    String csv = "";
    for (Text val : values) {
        if (csv.length() > 0) csv += ",";
        csv += val.toString();
    }
    context.write(key, new Text(csv));
}
```

Concatenación de valores en una lista separada por comas.

- Driver Configuración:
- `job.setMapperClass(MapClass.class)`
- `job.setReducerClass(Reducer.class)`
- `job.setOutputKeyClass(Text.class)`
- `job.setInputFormatClass(TextInputFormat.class)`

Compilación y Empaquetado (Build)

Transformando código fuente en ejecutable distribuido

1. Configurar Classpath

```
export HADOOP_CLASSPATH=$JAVA_HOME/lib/tools.jar
```

Permite a Java encontrar las librerías de Hadoop.

2. Compilar

```
hadoop com.sun.tools.javac.Main MyJob.java
```

Genera los archivos .class (MyJob\$MapClass.class, etc).

3. Crear JAR

```
jar cvf MyJob.jar My*
```

```
agregando: MyJob.class ...  
agregando: MyJob$MapClass.class ...
```

Empaqueta todas las clases en un único archivo.



Ejecución del JAR Nativo

```
hadoop jar MyJob.jar MyJob /practicas/cite75_99.txt /resultado7
```

El lanzador

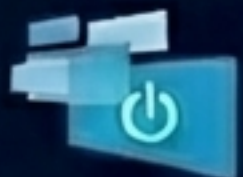
El artefacto
(paquete)

La clase Main
(Driver)

Argumento 0
(Input)

Argumento 1
(Output)

Si este comando finaliza con éxito, ha validado la capacidad completa de procesamiento distribuido (MapReduce V2) de su cluster.



Verificación de Resultados

Inspeccionar el contenido generado en HDFS.

```
$ hdfs dfs -cat /resultado7/part-r-00000 | head -5
```

```
3600000      3859201,3902183,4012392
```

```
3600001      3729810,3820100
```

```
3600002      3681203,3700101,3810292,3912001
```

```
...
```

El resultado muestra la 'Clave' (Patente citada) seguida de la lista concatenada de patentes que la citan.



Validación Completa del Cluster



Infraestructura Base (Shell Streaming) - I/O y YARN básico verificado.



Entorno Scripting (Python) - Distribución de ficheros y entornos locales verificados.



Desarrollo Nativo (Java) - Toolchain de compilación y Classpath verificados.

Su cluster Hadoop está operativo y listo para cargas de trabajo reales.

Siguientes pasos: Modifique `MyJob.java` para realizar analíticas más complejas sobre el dataset de patentes.